

Managing Iterative Processes in Software Project Management

1. What is an Iterative Process?

An **iterative process** is a software development approach in which the software is developed through **repeated cycles (iterations)**.

Instead of trying to develop the complete software in one cycle, the team:

Plans → Develops → Tests → Reviews → Gets Feedback → Improves → Repeats

Each iteration produces an improved version of the software.

Simple Example

Suppose we are developing an **Online Shopping System**.

Instead of developing everything together:

- **Iteration 1:** Login & Registration
- **Iteration 2:** Product Search & Product Details
- **Iteration 3:** Shopping Cart
- **Iteration 4:** Payment
- **Iteration 5:** Order Tracking

Each iteration adds functionality to the previous version.

2. What Does "Managing Iterative Processes" Mean?

In **Software Project Management (SPM)**, managing an iterative process means **planning, organizing, monitoring, controlling, and reviewing each iteration** so that the project progresses toward its final objectives.

The project manager must ensure:

- The right work is selected for each iteration.
- The iteration finishes within the planned time.
- Resources are available.
- Risks are controlled.
- Quality is maintained.
- Customer feedback is incorporated.

- The project remains within budget and schedule.

3. Why Use an Iterative Process?

Traditional development may follow:

Requirements → Design → Coding → Testing → Final Delivery

The customer may not see the actual working system until very late.

An iterative approach provides:

```
Iteration 1 → Feedback
      ↓
Iteration 2 → Feedback
      ↓
Iteration 3 → Feedback
      ↓
Iteration 4 → Final Product
```

Major Benefits

- Early delivery of working software.
- Early identification of defects.
- Easier handling of changing requirements.
- Continuous customer involvement.
- Better risk management.
- Better estimation as the project progresses.

4. Iterative Process Management Cycle

A project manager generally manages iterations through the following activities:

```
Iteration Planning
      ↓
Task Allocation
      ↓
Development
      ↓
Testing
      ↓
Monitoring
      ↓
Review / Demonstration
      ↓
Customer Feedback
      ↓
Retrospective
```

↓
Plan Next Iteration

This cycle continues until the required product is completed.

5. Step 1 — Iteration Planning

The first step is to decide **what should be developed during the iteration**.

The project manager and team consider:

- Product requirements
- Customer priorities
- Available resources
- Previous iteration results
- Technical dependencies
- Risks
- Estimated effort

Example

Suppose the product backlog contains:

Requirement	Priority
Login	High
Product Search	High
Cart	High
Dark Mode	Low
AI Recommendations	Low

For the next iteration, the team may select:

Login + Product Search

because they have higher priority.

6. Step 2 — Define Iteration Goals

Each iteration should have a **clear and measurable goal**.

Poor Goal:

Improve the application.

Good Goal:

Implement and test user registration and login functionality.

A clear goal helps the project manager determine whether the iteration was successful.

7. Step 3 — Break Work into Tasks

Large requirements are divided into smaller tasks.

Example: Login Feature

```
Login Feature
↓
UI Design
↓
Database Design
↓
Backend API
↓
Authentication
↓
Frontend Integration
↓
Testing
```

Breaking work into smaller tasks makes it easier to:

- Estimate effort.
 - Assign responsibilities.
 - Track progress.
 - Identify delays.
-

8. Step 4 — Estimate Effort

The team estimates how much effort each task requires.

Common estimation techniques include:

- Story points
- Expert judgment
- Planning poker
- Function points
- Historical data

Example

Task	Estimated Effort
Login UI	4 hours
Login API	6 hours
Database	3 hours
Testing	4 hours

Total estimated effort = **17 hours**

The project manager compares this with the team's available capacity.

9. Step 5 — Resource Allocation

The project manager assigns tasks according to:

- Skills
- Availability
- Experience
- Workload

Example

Team Member	Task
Developer 1	Frontend
Developer 2	Backend
Database Developer	Database
Tester	Testing

The manager must avoid overloading one team member while others have little work.

10. Step 6 — Development

The team starts implementing the selected features.

During development:

- Developers write code.
- Designers create interfaces.
- Database developers create required structures.
- Testers prepare test cases.

The project manager monitors progress without unnecessarily interfering with technical work.

11. Step 7 — Continuous Testing

Testing should occur **throughout the iteration**, not only at the end of the entire project.

Testing may include:

- Unit testing
- Integration testing
- Functional testing
- Regression testing
- User acceptance testing

Why?

If a defect is discovered early, it is generally easier to fix than discovering it several iterations later.

12. Step 8 — Monitor Iteration Progress

The project manager continuously checks:

Schedule

Is the team on time?

Cost

Are we staying within the budget?

Scope

Are we developing only the agreed features?

Quality

Are defects increasing?

Resources

Are team members available?

Risks

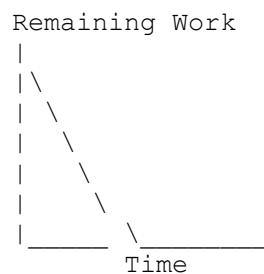
Have new risks appeared?

13. Progress Tracking

Various tools can be used to monitor progress.

Burndown Chart

A **burndown chart** shows how much work remains during an iteration.



Ideally, remaining work should decrease as the iteration progresses.

14. Step 9 — Managing Changes

One of the biggest advantages of iterative development is the ability to handle changing requirements.

However, **every change should not automatically be accepted.**

The project manager should evaluate:

- Business value
- Cost
- Time impact
- Technical impact
- Risk
- Dependencies

Example

Customer requests:

"Add online payment during the current iteration."

The manager evaluates whether:

- The team has capacity.
- Payment affects existing modules.
- Security requirements are satisfied.
- The change can be completed without affecting the iteration goal.

If not, it may be moved to the next iteration.

15. Step 10 — Iteration Review

At the end of an iteration, the team demonstrates the completed functionality to stakeholders.

Review Questions

- What was completed?
- What was not completed?
- Does the feature meet requirements?
- What does the customer think?
- What should be changed?

Example

The team demonstrates the Login module.

Customer says:

"Login works, but we also need OTP-based authentication."

The requirement can then be added or prioritized for a future iteration.

16. Step 11 — Customer Feedback

Customer feedback is one of the most important parts of iterative development.

Feedback may result in:

- New requirements.
- Requirement changes.
- UI improvements.
- Bug fixes.
- Priority changes.

The updated requirements become input for future iterations.

17. Step 12 — Iteration Retrospective

A **retrospective** is a meeting where the team evaluates its own working process.

Three common questions are:

What went well?

Example:

Testing was completed early.

What did not go well?

Example:

Backend development was delayed.

What should we improve?

Example:

Start API development earlier in the next iteration.

The goal is **continuous process improvement**.

18. Managing Risks in Iterative Processes

Iterative development helps identify risks early.

Example

Suppose a project requires a new payment technology.

Instead of waiting until the final stage, the team can test the payment technology in an early iteration.

If it fails:

- The problem is discovered early.
- Alternative technology can be investigated.
- Project risk is reduced.

Common Risks

- Technical risks
 - Requirement changes
 - Resource shortage
 - Schedule delays
 - Integration problems
 - Security problems
 - Performance problems
-

19. Managing Scope

Scope management is extremely important.

Without proper control, continuous changes can cause **scope creep**.

Example

Original requirement:

Build an online shopping website.

Customer gradually asks for:

- Chat system
- AI recommendation
- Voice search
- Loyalty points
- Live tracking
- Social media integration

If everything is immediately added, the project may never finish.

Therefore, the project manager prioritizes requirements and decides **what belongs in the current iteration and what should be postponed**.

20. Managing Quality

Quality must be maintained in every iteration.

The project manager should ensure:

- Coding standards are followed.
- Testing is performed.

- Defects are tracked.
- Code reviews are performed.
- Acceptance criteria are satisfied.

Important Principle

Do not postpone quality until the final iteration.

21. Managing Communication

Iterative projects require frequent communication between:

- Project manager
- Developers
- Testers
- Customers
- Business stakeholders

Communication may happen through:

- Daily meetings
- Sprint/iteration reviews
- Progress reports
- Issue trackers
- Project management tools
- Team discussions

Good communication helps identify problems early.

22. Managing Dependencies

Some tasks cannot start until another task is completed.

Example

```
Database Design
  ↓
Backend API
  ↓
Frontend Integration
  ↓
Testing
```

The project manager must identify such dependencies and plan accordingly.

Otherwise, a delay in one task can delay the entire iteration.

23. Managing Multiple Iterations

The project manager must also manage the **overall project**, not just individual iterations.

Example:

Iteration	Goal	Status
1	Login	Completed
2	Product Catalog	Completed
3	Cart	In Progress
4	Payment	Planned
5	Order Tracking	Planned

The manager continuously compares actual progress with the overall project plan.

24. Definition of Done

Before declaring an iteration complete, the team should define what "**Done**" means.

For example, a feature is considered complete only when:

- Coding is completed.
- Code review is completed.
- Unit tests pass.
- Integration testing is completed.
- Major defects are fixed.
- Customer acceptance criteria are satisfied.
- Documentation is updated where required.

This prevents the team from claiming incomplete work as finished.

25. Example: College Management System

Suppose a college wants to develop a **College Management System**.

Iteration 1

Goal: Student Registration

Develop:

- Registration form
- Database
- Validation
- Testing

Customer reviews it.

Iteration 2

Goal: Attendance

Develop:

- Attendance entry
- Attendance calculation
- Reports

Customer provides feedback.

Iteration 3

Goal: Examination

Develop:

- Marks entry
 - Result calculation
 - Result report
-

Iteration 4

Goal: Fee Management

Develop:

- Fee records
- Payment
- Receipts

Each iteration produces a working improvement to the system.

26. Advantages of Managing Iterative Processes

1. Early Delivery

Working functionality becomes available early.

2. Early Risk Identification

Problems are discovered sooner.

3. Better Requirement Management

Changing requirements can be incorporated.

4. Continuous Customer Feedback

The product is continuously reviewed.

5. Better Quality

Testing happens repeatedly.

6. Improved Estimation

Future iterations can be estimated using actual project performance.

7. Better Visibility

Stakeholders can see actual progress rather than relying only on reports.

27. Challenges

Managing iterative processes also has challenges:

- Frequent requirement changes.
- Difficulty maintaining long-term schedules.
- Scope creep.
- Continuous customer involvement required.
- Team coordination.

- Repeated testing and integration.
 - Managing technical debt.
 - Maintaining architecture while requirements evolve.
-

28. Traditional vs Iterative Project Management

Traditional	Iterative
Plan most things upfront	Plan progressively
Large final delivery	Frequent smaller deliveries
Changes are difficult	Changes are expected
Feedback often comes late	Feedback comes frequently
Testing may occur later	Continuous testing
Risk may be discovered late	Risks identified earlier
Progress based heavily on planned tasks	Progress demonstrated through working software

29. Important Terms to Remember

Term	Meaning
Iteration	A development cycle that produces an improved product
Iteration Goal	What the team intends to achieve
Backlog	Prioritized list of work
Timebox	Fixed period for completing work
Increment	Working addition to the product
Review	Evaluation of completed work
Retrospective	Evaluation of the development process
Feedback	Input used to improve future iterations
Definition of Done	Conditions required to consider work complete
Scope Creep	Uncontrolled expansion of project requirements